Project: Auto-scaling Data Access and Business Logic

Due: 04/25/2026 12:59 AM
Extended Deadline: 04/29/2026 10:53 AM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Single Instance - MySQL
- Task Overview
- Task

Separate Reader + Writer for MySQL
- Task Overview
- Task

Horizontal Autoscaling
- Task Overview
- Task

Wrap Up
- Conclusion

Single Instance - MySQL (Task)

The Story Continues

After impressing your boss with your LLM API server, you have become the go-to person for Cloud Native development services in your organization. You handle stateless services very well, and have become the go-to person when it comes to developing and building stateless systems that scale.
One night, you wake up in a cold sweat realizing that your system isn't that reliable. In particular, your existing login service is not scalable, as it is stateful. We would like to scale this system up, and in doing so start to pick at the ways that teams scale up stateful systems for production-grade systems.
As we will discuss in lecture, this project will focus on the development of stateful OLTP systems. These are systems where you will interact directly with a database, and care more about read and write performance as opposed to just read performance.

Intro

Up until now, your Django login service has been running with SQLite — a lightweight database that lives directly on the container's filesystem. This works fine for development, but it has a critical flaw: every time your pod restarts, all your data is wiped. In this task, you will replace SQLite with a production-grade MySQL database deployed to your cluster via Flux and Helm. You will then connect your Django app to MySQL, run database migrations, and use an initContainer to bootstrap the database on startup. By the end of this task, your login service will have a persistent, resilient data layer that survives pod restarts — the final piece needed to make your system production-ready.

Setup

This project assumes that your cluster state is exactly what you had for **Task 1 - Istio** of the project on `Service Mesh & Observability`. You will use the same handout you've been using from Unit 0 until Task 1 of the **Service Mesh & Observability** project.

You should already have your AKS cluster, ACR, Flux bootstrapped to your repo, Tekton building and pushing container images, and the Flux `Kustomization` pattern.

Commands for this subtask

```
flux reconcile source git flux-system

flux get kustomizations
```

Subtask 1: MySQL Helm Charts

In this subtask you will deploy MySQL to the cluster using Flux and Helm.

The repository contains two directories you need to work with:

- `helm-sources/` — defines where Helm charts are pulled from
- `helm-releases/` — defines what gets installed on the cluster

Step 1: Review the Helm Source

Open `helm-sources/mysql.yaml`. This file tells Flux where to pull the MySQL chart from (Bitnami's OCI registry). You do not need to modify this file.

```yaml
# helm-sources/mysql.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: bitnami
  namespace: flux-system
spec:
  interval: 1h
  url: oci://registry-1.docker.io/bitnamicharts
  type: oci
```

Step 2: Configure the Helm Release

```yaml
# helm-releases/mysql.yaml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: mysql
  namespace: default
spec:
  interval: 1h
  chart:
    spec:
      chart: mysql
      version: ">=9.0.0"
      sourceRef:
        kind: HelmRepository
        name: bitnami
        namespace: flux-system
  install:
    createNamespace: false
  values:
    image:
      registry: docker.io
      repository: bitnamilegacy/mysql
      tag: "9.4.0-debian-12-r1"
    auth:
      rootPassword: "password"
      database: "login"
      username: "my-user"
      password: "my-password"
    primary:
      persistence:
```

```
        enabled: true
        size: 10Gi
```

Warning

Whenever you are given Helm source and release files, you should add them to the `kustomization.yaml` file in their respective directories. This is how Flux knows which files to reconcile to the cluster.

Step 3: Push your changes

Once you are happy with your changes, commit and push to the repository:

```
git add helm-sources/mysql.yaml helm-releases/mysql.yaml
git commit -m "configure mysql helm release"
git push
```

Flux will automatically detect the changes and reconcile them to the cluster within a few minutes.

Step 4: Verify the deployment

Check that the HelmRelease was applied successfully:

```
flux get helmrelease mysql -n default

NAME    REVISION        SUSPENDED       READY   MESSAGE
mysql   14.0.3          False           True    Helm instal
```

Check that the MySQL pod is running:

```
kubectl get pods -n default

NAME                                READY   STATUS    RESTA
llmapp-deployment-7865d87cb-cx7rn   2/2     Running   0
login-ddcdd89d5-dffh8               2/2     Running   0
mysql-0                             2/2     Running   0
```

Step 5: Test the connection

Once the pod is running, test the connection:

```
kubectl run mysql-client --image=mysql:5.7 -it --rm --resta
  mysql -h mysql -uroot -ppassword -e 'SHOW DATABASES;'

+--------------------+
| Database           |
+--------------------+
| information_schema |
| login        |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
pod "mysql-client" deleted
```

You should see a list of databases including the one you configured in Step 2.

Subtask 2: Connecting Django to MySQL

Right now, your Django app is using **SQLite** as its database. SQLite is fine for local development, but we need to switch it to use the **MySQL pod** you deployed in the previous subtask.

What you need to do

1. Update Django's database settings

In your Django app's `settings.py`, replace the existing SQLite configuration with the following MySQL configuration:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'login',
        'USER': 'root',
        'PASSWORD': 'password',
        'HOST': 'mysql',
        'PORT': '3306',
    }
}
```

We use `mysql` as the host because the Django pod and the MySQL pod are running in the same Kubernetes cluster — this means Django can reach MySQL directly using its service name.

2. Update your Dockerfile

MySQL requires some system-level packages to be available inside your container. Open your `Dockerfile` and add the following before your Python install steps:

```
RUN apt-get update && apt-get install -y \
    python3-dev \
    default-libmysqlclient-dev \
    build-essential \
    pkg-config

RUN pip install --no-cache-dir -r requirements.txt
```

3. Rebuild and redeploy

Commit and push your changes to GitHub.

Recall that you have already set up a complete CI/CD platform that builds, test and deploys your container images.
With that knowledge, trigger the pipeline run by running:

```
kubectl create -f pipelinerun.yaml
tkn pipeline logs -f -L

tkn pipelinerun describe -L
```

Debugging Needed For Pipeline Run Error

If you run into the following error:

```
  File "/usr/local/lib/python3.12/site-packages/django/db/bac
    raise ImproperlyConfigured(
        "Error loading MySQLdb module.\nDid you install mysql
    ) from err
django.core.exceptions.ImproperlyConfigured: Error loading My
Did you install mysqlclient?
```

Think carefully about how such an error would occur and how you might go about resolving it.

Hints

We have confirmed this works with the `python:3.12` base image. If you run into issues with other base images, let us know.

Subtask 3: Initializing your Django System

While our MySQL database was initialized using the `auth` section of the HelmRelease, Django requires a different approach. We need to run database migrations and create a superuser before the main Django container starts. For this we will use **initContainers**.

initContainers are containers that run and must complete successfully before the main application container starts. They are ideal for one-time setup tasks like database migrations.

Step 1: Update your HelmRelease

Add an `initContainer` to your Django deployment file in **k8s/login/deployment.yaml** that runs migrations and creates a superuser:

```
initContainers:
    - name: django-init
    image: <YOUR-ACR>.azurecr.io/login:<UNIQUE-HASH> #
    command:
    - /bin/sh
    - -c
    - python manage.py migrate && python manage.py crea
    env:
    - name: DJANGO_SUPERUSER_PASSWORD
        value: "asdasd"
```

The initContainers section must be at the same level as containers under spec.

Step 2: Push and reconcile

Commit and push your changes and let Flux reconcile:

What this does

1. Runs `python manage.py migrate` to apply any pending database migrations.
2. Runs `python manage.py createsuperuser` to create an admin user. The `|| true` ensures the command does not fail if the superuser already exists.
3. Once the initContainer completes successfully, the main Django container starts.

The key benefit of this approach is that the initContainer only runs during pod startup — if the main Django container crashes and restarts, the initContainer does not re-run.

Final Validation

Assuming you have done all of this as expected, you can now check that your system works by:

1. Switching your **Django service** from **ClusterIP** to **LoadBalancer**
2. Running the following to get a simple **OAuth Token**:

```
curl -H 'Content-Type: application/json' -d '{"username":"a
```

Submit Your Work

For this assignment, we will do the following to check that your system works:

1. Check that the **existing external service** can connect to your system, end to end. For this reason, you will need to make sure that you have the expected username and password, and that **your Django service** is the **only** service that is exposed externally.
2. Check that **your Django application** is connected to the **MySQL** service you created, and that **the tables have the expected user**. In order to test our autograder access works, make sure the following command runs:

```
kubectl exec deploy/login -it -- python manage.py shell -c

12 objects imported automatically (use -v 2 for details).

('auth_group',)
```

To submit, use:

```
./submitter -t task1
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform